

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO CUỐI KỲ
HỌC SÂU

Người hướng dẫn: **GV Lê Anh Cường**

Người thực hiện: **Nguyễn Mỹ Duyên - 52200190**

Nguyễn Minh Khánh - 52200187

Phan Văn Quốc Dinh - 52200209

Khóa: 26

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2025

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO CUỐI KỲ

HỌC SÂU

Người hướng dẫn: **GV Lê Anh Cường**

Người thực hiện: **Nguyễn Mỹ Duyên - 52200190**

Nguyễn Minh Khánh - 52200187

Phan Văn Quốc Dinh - 52200209

Khóa: 26

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2025

LỜI CẢM ƠN

Trong quá trình thực hiện bài báo cáo, chúng em đã được Thầy Lê Anh Cường hỗ trợ về mặt kiến thức và phương pháp nghiên cứu. Thầy đã giúp chúng em hiểu rõ hơn về yêu cầu của bài báo cáo và đưa ra những ý kiến xây dựng giúp chúng em nâng cao chất lượng bài báo cáo của chúng em. Ngoài ra, Thầy Cường còn tạo điều kiện để em có thể thực hiện bài báo cáo này một cách hiệu quả hơn.

Chúng em rất cảm kích và biết ơn thầy trong quá trình học tập môn Học Sâu. Thầy đã rất tận tình, tâm huyết giúp đỡ và hướng dẫn chúng em trong suốt quá trình học tập, từ đó giúp chúng em hiểu rõ hơn về những khái niệm cơ bản và tạo ra được một cách suy nghĩ toán học sâu sắc.

Và đặc biệt hơn là chúng em cũng muốn bày tỏ lòng biết ơn đến Trường Đại học Tôn Đức Thắng đã cung cấp cho chúng em một môi trường học tập và nghiên cứu chuyên nghiệp, đồng thời hỗ trợ chúng em về các tài liệu nghiên cứu để thực hiện bài báo cáo.

ĐỒ ÁN ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Chúng tôi xin cam đoan đây là sản phẩm của riêng chúng tôi và được sự hướng dẫn của Thầy Lê Anh Cường. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong báo cáo còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung báo cáo của chúng tôi. Trường đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày tháng năm

Tác giả

(ký tên và ghi rõ họ tên)



Nguyễn Minh Khánh



Nguyễn Mỹ Duyên



Phan Văn Quốc Dinh

PHẦN XÁC NHẬN VÀ ĐÁNH GIÁ CỦA GIẢNG VIÊN

Phần xác nhận của GV hướng dẫn

Tp. Hồ Chí Minh, ngày tháng năm
(kí và ghi họ tên)

Phần đánh giá của GV chấm bài

Tp. Hồ Chí Minh, ngày tháng năm
(kí và ghi họ tên)

TÓM TẮT

Chương 1 – Giới thiệu

Trình bày bối cảnh, động lực và mục tiêu nghiên cứu kết hợp hai hướng: khảo sát lý thuyết về attention và triển khai mô hình OCR. Xác định phạm vi dữ liệu (ICDAR2015 word images), yêu cầu hệ thống (Python, PyTorch, Colab GPU) và phương pháp nghiên cứu. Đưa ra kết cấu báo cáo gồm sáu chương chính.

Chương 2 – Cơ sở lý thuyết về Attention

Tổng quan kiến trúc Transformer, định nghĩa Q–K–V và Positional Encoding. Phân tích năm biến thể attention (Self-Attention, MQA, GQA, FlashAttention v2, RoPE) về nguyên lý, độ phức tạp, ưu-nhược điểm và ứng dụng trong các mô hình ngôn ngữ lớn (LLM).

Chương 3 – Cài đặt minh họa Attention

Triển khai ví dụ cho từng cơ chế attention: hiển thị ma trận trọng số, giải thích cách token tương tác và so sánh hành vi giữa các biến thể. Cho phép trực quan hóa và hiểu sâu cách mỗi attention hoạt động.

Chương 4 – Lý thuyết mô hình OCR CNN + Transformer

Phân tích kiến trúc CNN (ResNet-18) để trích xuất feature map và cấu trúc Transformer Decoder để sinh chuỗi văn bản. Trình bày cơ chế cross-attention giữa encoder và decoder, hàm mất mát, teacher forcing và optimizer.

Chương 5 – Thực nghiệm & Đánh giá OCR

Mô tả chi tiết quy trình thực nghiệm trên Google Colab: chuẩn bị dữ liệu, xây dựng Dataset/DataLoader, huấn luyện mô hình với Adam. Đánh giá bằng exact match accuracy và CER/WER, minh họa kết quả (ảnh gốc và văn bản dự đoán), trực quan hóa heatmap attention và phân tích hạn chế.

Chương 6 – Kết luận & Hướng phát triển

Tổng kết đóng góp của hai bài nghiên cứu, rút ra bài học kinh nghiệm và chỉ ra hạn chế. Đề xuất hướng nghiên cứu tiếp theo

MỤC LỤC

LỜI CẢM ƠN	i
PHẦN XÁC NHẬN VÀ ĐÁNH GIÁ CỦA GIẢNG VIÊN	iii
TÓM TẮT	iv
MỤC LỤC.....	1
CHƯƠNG 1: GIỚI THIỆU	3
1.1 Lý do chọn đề tài.....	3
1.2 Mục đích của đề tài	3
1.4 Phạm vi chọn đề tài.....	3
1.4.1 Phạm vi dữ liệu	3
1.4.2 Yêu cầu hệ thống	3
1.5 Phương pháp nghiên cứu	4
1.6 Kết cấu báo cáo	4
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VỀ ATTENTION (Bài 1).....	5
2.1 Tổng quan Transformer & Attention	5
2.2 Các khái niệm nền tảng.....	5
2.3 Phân loại & phân tích cơ chế Attention	5
2.3.1 Self-Attention.....	5
2.3.2 Multi-Query Attention (MQA)	6
2.3.3 Grouped-Query Attention	6
2.3.4 FlashAttention v2.....	7
2.3.5 RoPE Attention	7
2.4 Ứng dụng trong các mô hình LLM hiện đại	7
CHƯƠNG 3: CÀI ĐẶT MINH HOẠ ATTENTION	9
3.1 Self-Attention.....	9
3.2 MQA	10
3.3 GQA.....	12
3.4 FlashAttention v2.....	14
3.5 RoPE	16
CHƯƠNG 4: LÝ THUYẾT MÔ HÌNH OCR CNN + TRANSFORMER.....	19
4.1 Kiến trúc CNN cho trích đặc trưng ảnh	19

4.1.1 Lớp Convolution, Activation và Pooling.....	19
4.2 Kiến trúc Transformer-Decoder.....	19
4.2.1 Cấu trúc lớp Decoder	20
4.3 Cơ chế Attention giữa đặc trưng ảnh và trình giải mã	20
4.4 Hàm mất mát và quy trình huấn luyện.....	20
4.4.1 Cross-Entropy Loss.....	20
4.4.2 Teacher Forcing và Optimizer	21
CHƯƠNG 5: THỰC NGHIỆM & ĐÁNH GIÁ OCR	22
5.1 Thiết lập môi trường và dữ liệu	22
5.1.1 Bộ ảnh kiểm thử.....	22
5.1.2 Công cụ, thư viện và môi trường	22
5.2 Nguyên lý kiến trúc CNN + Transformer Decoder	23
5.2.1 Vai trò của CNN trong OCR.....	23
5.2.2 Cơ chế hoạt động của Transformer Decoder	24
5.2.3 Attention giữa đặc trưng ảnh và decoder	25
5.3 Triển khai mô hình đơn giản.....	26
5.3.1 Chuẩn bị dữ liệu và tiền xử lý.....	26
5.3.2 Xây dựng CNN Encoder	27
5.3.3 Cấu tạo Transformer Decoder.....	27
5.3.4 Kết hợp Encoder–Decoder.....	27
5.3.5 Huấn luyện và điều chỉnh siêu tham số	28
5.4 Đánh giá và trình bày kết quả	28
5.4.1 Đánh giá độ chính xác trên tập Test	28
5.4.2 Minh họa kết quả đầu ra	29
CHƯƠNG 6: KẾT LUẬN & HƯỚNG PHÁT TRIỂN	31
6.1 Kết luận chính	31
6.2 Hạn chế & bài học kinh nghiệm	31
TÀI LIỆU THAM KHẢO.....	33

CHƯƠNG 1: GIỚI THIỆU

1.1 Lý do chọn đề tài

Cơ chế attention đã trở thành thành phần không thể thiếu trong các mô hình ngôn ngữ. Đồng thời, việc **trích xuất văn bản từ hình ảnh** (scene-text OCR) ngày càng quan trọng trong các ứng dụng như đọc hóa đơn tự động, hỗ trợ người khiếm thị, và chuyển đổi tài liệu in sang dạng số. Do đó, đề tài tập trung vào:

- Khảo sát và minh họa năm biến thể attention hiện đại (Self-Attention, MQA, GQA, FlashAttention v2, RoPE) để làm rõ ưu – nhược trong các LLM.
- Triển khai mô hình OCR kết hợp CNN + Transformer Decoder từ đầu, nhằm hiểu sâu quá trình trích xuất đặc trưng ảnh và sinh chuỗi văn bản.

1.2 Mục đích của đề tài

- **Nghiên cứu lý thuyết** về attention: phân tích nguyên lý, độ phức tạp, ứng dụng trong các mô hình LLM hàng đầu.
- **Triển khai minh họa** từng cơ chế attention trên ví dụ để trực quan hóa ma trận weights.
- **Xây dựng và đánh giá** một mô hình OCR end-to-end trên bộ dữ liệu ICDAR2015, từ preprocessing, encoder ResNet-18 đến decoder Transformer.

1.4 Phạm vi chọn đề tài

1.4.1 Phạm vi dữ liệu

- **Bài 1 (Attention):** chỉ sử dụng các ví dụ chuỗi ngắn và minh họa ma trận attention; không yêu cầu corpus song ngữ lớn.
- **Bài 2 (OCR):** tập trung bộ dữ liệu ICDAR 2015 Challenge 4 word images (ảnh quảng cáo, hóa đơn, biển hiệu); tách Train/Test 80/20; không mở rộng sang handwriting hay ngôn ngữ khác.

1.4.2 Yêu cầu hệ thống

- **Phần mềm:** Python 3.x, thư viện PyTorch & torchvision, Pillow, scikit-learn, unidecode, Matplotlib.
- **Phần cứng:** GPU (Colab/Google Colab với CUDA), RAM ≥ 8 GB.

- **Môi trường:** Google Colab để đảm bảo khả năng tái lập (consistency) và truy cập GPU miễn phí.

1.5 Phương pháp nghiên cứu

- **Thu thập tài liệu:** khảo sát bài báo, blog, và mã nguồn mở về các biến thể attention.
- **Phân tích lý thuyết:** tổng hợp khái niệm Q–K–V, PE, MHA, so sánh ưu nhược.
- **Thiết kế minh họa:** cài đặt module cho từng attention, trực quan hóa ma trận weights.
- **Xây dựng OCR pipeline:** tiền xử lý ảnh & nhãn, định nghĩa encoder/decoder, huấn luyện và đánh giá.
- **Đánh giá kết quả:** sử dụng metric exact match, CER/WER, vẽ heatmap attention, phân tích hạn chế.
- **Tổng kết & đề xuất:** rút ra bài học, hướng phát triển, đề xuất cải tiến cho nghiên cứu tương lai.

1.6 Kết cấu báo cáo

Báo cáo gồm **06 chương** chính:

- **Chương 1:** Giới thiệu — Lý do chọn đề tài, mục tiêu, phạm vi, phương pháp và kết cấu.
- **Chương 2:** Cơ sở lý thuyết về Attention — Tổng quan Transformer, Q–K–V, các biến thể attention và ứng dụng.
- **Chương 3:** Cài đặt minh họa Attention — Mô-đun cho Self-Attention, MQA, GQA, FlashAttention v2, RoPE.
- **Chương 4:** Lý thuyết mô hình OCR CNN + Transformer — Kiến trúc encoder CNN, decoder Transformer, cơ chế cross-attention.
- **Chương 5:** Thực nghiệm & Đánh giá OCR — Thiết lập dữ liệu, triển khai mô hình, metric đánh giá, minh họa kết quả, phân tích.
- **Chương 6:** Kết luận & Hướng phát triển — Tổng kết đóng góp, hạn chế, bài học và định hướng nghiên cứu tiếp theo.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VỀ ATTENTION (Bài 1)

Chương này tổng quan Transformer, định nghĩa Query–Key–Value và trình bày sâu năm cơ chế attention hiện đại, làm nền tảng cho phần cài đặt. Mỗi cơ chế được phân tích ý tưởng, độ phức tạp, ưu – nhược, so sánh với self-attention gốc và nêu ứng dụng LLM.

2.1 Tổng quan Transformer & Attention

Kiến trúc Transformer loại bỏ hoàn toàn sự phụ thuộc tuần tự của RNN, thay thế bằng **self-attention** cho phép mô hình nắm bắt ngữ cảnh toàn cục và song song hoá hiệu quả trên GPU. Một khối Transformer cơ bản gồm các tầng Multi-Head Attention (MHA), Positional Encoding và Feed-Forward Network. Chính MHA – thông qua nhiều attention – giúp mô hình học các quan hệ ngữ nghĩa đa dạng giữa các token. Bên cạnh đó, cơ chế **Residual Connection** và **Layer Normalization** đảm bảo việc huấn luyện sâu, ổn định.

2.2 Các khái niệm nền tảng

- Query (Q): Vector đại diện cho từ cần tính toán trọng số Attention. Key (K): Vector dùng để so sánh với Query, xác định mức độ liên quan. Value (V): Vector chứa thông tin thực tế được tổng hợp dựa trên trọng số Attention.
- Positional Encoding (PE): Vì Transformer thiếu cơ chế tuần tự tự nhiên, PE bổ sung thông tin vị trí cho từng token, thường dưới dạng sóng sin – cos (absolute PE) hoặc xoay tọa độ (Rotary PE).
- Multi-Head Mechanism: Thay vì một ánh xạ duy nhất, mô hình sinh nhiều cặp Q/K/V song song, giúp khám phá quan hệ ở các không gian riêng biệt và sau đó ghép nối (concatenate) để tăng độ biểu đạt.

2.3 Phân loại & phân tích cơ chế Attention

2.3.1 Self-Attention

Self-attention đầy đủ tính mọi cặp tương tác Q–K trong chuỗi, tạo ma trận chú ý $n \times n$ (với n là độ dài chuỗi). Độ phức tạp $O(n^2 \cdot d)$ (d : kích thước embedding) bảo đảm biểu diễn ngữ cảnh hoàn chỉnh nhưng chi phí bộ nhớ cao khi n lớn. Self-attention là nền tảng của GPT-3, BERT và hầu hết các LLM tiêu chuẩn.

Nguyên lý	Tính mọi cặp Q–K, tạo ma trận chú ý $n \times n$
Độ phức tạp	$O(n^2 \cdot d)$ tính toán & bộ nhớ
Ưu điểm	Nắm bắt toàn bộ ngữ cảnh, phụ thuộc dài
Hạn chế	Bộ nhớ & thời gian $\uparrow$ bậc hai với n
Ứng dụng	BERT, GPT-3, T5

2.3.2 Multi-Query Attention (MQA)

MQA duy trì nhiều head Query độc lập song **dùng chung** Key–Value cho tất cả heads. Bằng cách đó, mô hình giảm một nửa bộ nhớ cho K/V, rút ngắn thời gian suy luận trong khi vẫn bảo lưu tính đa dạng của Q. Google PaLM và một số phiên bản ChatGPT tối ưu hoá inference đã áp dụng MQA.

Nguyên lý	Q riêng cho mỗi head, dùng chung một cặp K/V
Độ phức tạp	$O(n^2 \cdot d)$; giảm $\approx 50\%$ bộ nhớ K/V
Ưu điểm	Giảm truy xuất RAM GPU, tăng tốc inference
Hạn chế	Đa dạng K/V giảm, độ chính xác giảm nhẹ
Ứng dụng	Google PaLM, GPT-3.5 Turbo

2.3.3 Grouped-Query Attention

GQA là bước trung gian giữa self-attention đầy đủ và MQA: các head được chia thành g nhóm, mỗi nhóm dùng chung K/V. Thiết kế này cân bằng chi phí bộ nhớ và khả năng biểu diễn; khi $g = h$ (số head) GQA thu về self-attention gốc, khi $g = 1$ trở thành MQA. Mô hình OPT-175B (Meta AI) ứng dụng GQA để mở rộng lên hàng trăm tỷ tham số mà vẫn phù hợp tài nguyên GPU.

Nguyên lý	Chia h head thành g nhóm, mỗi nhóm chung K/V
Độ phức tạp	$O(n^2 \cdot d / g)$
Ưu điểm	Cân bằng tốc độ & chất lượng

Hạn chế	Cần chọn g hợp lý; g quá nhỏ ↓ chính xác
Ứng dụng	Falcon-180B, LLaMA-2

2.3.4 FlashAttention v2

FlashAttention (Tri Dao et al., 2023) tái cấu trúc vòng tính softmax thành các khối chặn (blockwise) tối ưu bộ nhớ đệm và băng thông, giữ nguyên công thức self-attention nhưng tăng thông lượng 2–3×, giảm đáng kể bộ nhớ tạm. Phiên bản v2 bổ sung hỗ trợ BF16, cú pháp PyTorch native và kernel CUDA tùy chỉnh, hiện đã trở thành chuẩn de-facto trong huấn luyện LLM tốc độ cao.

Nguyên lý	Tiling + kernel fusion; Softmax & matmul theo khối
Độ phức tạp	$O(n^2 \cdot d)$ nhưng ↓ 50–70 % bộ nhớ tạm
Ưu điểm	↑ thông lượng 2–4×, hỗ trợ chuỗi > 16 K
Hạn chế	Phụ thuộc kernel CUDA/ROCm, cần BF16/FP16
Ứng dụng	HF Transformers, NVIDIA NeMo, Mosaic MPT

2.3.5 RoPE Attention

RoPE không thêm vector vị trí vào embedding mà **xoay** (rotate) trục của Q/K theo hàm tuần tự sin-cos, bảo toàn tính bất biến theo tịnh tiến và cho phép mô hình ngoại suy sang chuỗi dài hơn. RoPE duy trì độ phức tạp $O(n^2)$ tương tự self-attention nhưng cải thiện khả năng tổng quát vị trí; GPT-NeoX, MPT và LLaMA đồng loạt lựa chọn kỹ thuật này.

Nguyên lý	Xoay Q/K trong mặt phẳng phức (sin-cos) để mã hoá vị trí
Độ phức tạp	$O(n^2 \cdot d)$ (như Self-Attention)
Ưu điểm	Giữ tương quan vị trí, ngoại suy chuỗi dài
Hạn chế	Khó phối hợp với PE khác hoặc attention tuyến tính
Ứng dụng	GPT-NeoX-20B, LLaMA-v1/v2, MPT-7B

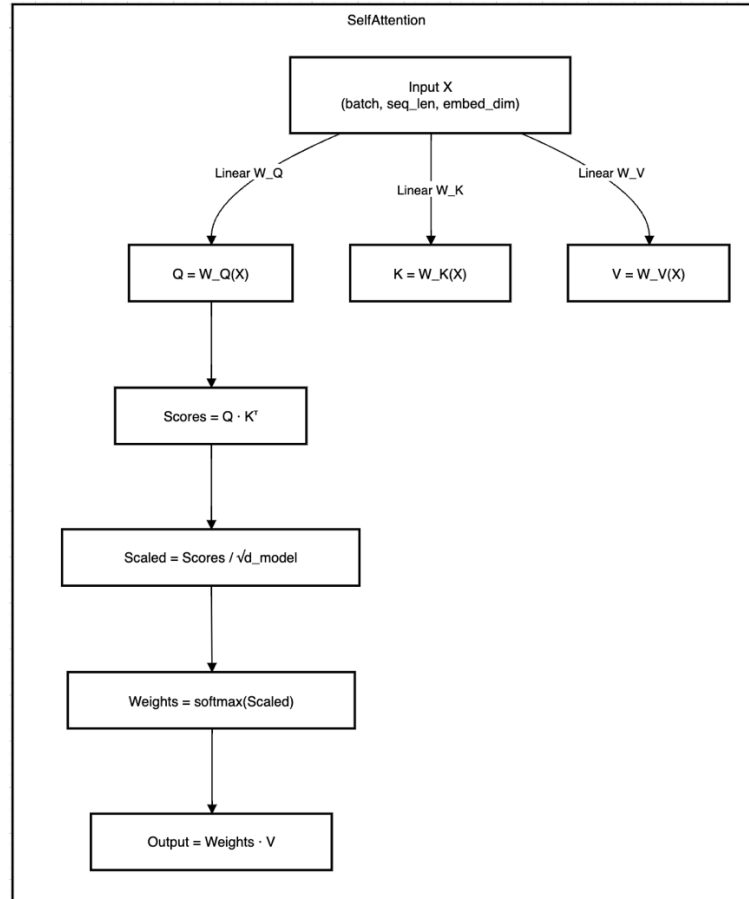
2.4 Ứng dụng trong các mô hình LLM hiện đại

- **Self-Attention đầy đủ:** vẫn được giữ nguyên trong GPT-3, GPT-4 và T5-XXL để bảo đảm chất lượng sinh văn bản cao nhất, chấp nhận chi phí n^2 nhờ cụm GPU/TPU quy mô siêu lớn.
- **Multi-Query Attention (MQA):** Google **PaLM-2** và kernel inference của **GPT-3.5 Turbo** dùng MQA để cắt $\sim 50\%$ bộ nhớ Key/Value, tăng tốc phục vụ lên $1.3\text{--}1.6\times$ mà hầu như không giảm độ chính xác.
- **Grouped-Query Attention (GQA):** **LLaMA-2-70B** và **Falcon-180B** chia head thành nhóm ($g \approx 8$) nhằm cân bằng tốc độ và chất lượng, cho phép chạy mô hình $\geq 70\text{ B}$ tham số trên GPU 80 GB.
- **FlashAttention v2:** các dòng **Mosaic MPT-30B**, **NVIDIA NeMo Megatron-GPT** và gần như toàn bộ mô hình trên HuggingFace (khi bật `--flash-attn`) khai thác kernel này để tăng $2\text{--}4\times$ thông lượng huấn luyện, giảm $50\text{--}70\%$ bộ nhớ tạm và xử lý chuỗi trên 16 K token.
- **Rotary Positional Embedding (RoPE):** **GPT-NeoX-20B**, **LLaMA v1/v2** và **Mistral/Mixtral** tích hợp RoPE để ngoại suy vị trí, giúp đọc ngữ cảnh tới 32 K token sau khi chỉ huấn luyện ở 4 K token.

CHƯƠNG 3: CÀI ĐẶT MINH HOẠ ATTENTION

3.1 Self-Attention

Module:



Input:

Đầu vào:
`tensor([[[[1., 0., 1., 0.],
[0., 2., 0., 2.],
[1., 1., 1., 1.]]]])`

Mã trận trọng số (2 head)

Mã trận Attention Weights:
`tensor([[[[0.3238, 0.3138, 0.3624],
[0.4046, 0.1633, 0.4321],
[0.3607, 0.2211, 0.4182]]], grad_fn=<SoftmaxBackward0>)`

Giải thích

- Mã trận weights có kích thước 3×3 , mỗi hàng cộng lại đúng 1 (do Softmax).

- Hàng 2, cột 3 = 0.4321 là giá trị lớn nhất toàn ma trận $\Rightarrow$ tại bước tính, token 2 “chú ý” mạnh nhất tới token 3.
- Các phần tử trên đường chéo (0.3238; 0.1633; 0.4182) cho thấy mức độ mỗi token tự chú ý vào chính nó; token 2 tự chú ý ít nhất vì có lựa chọn token 3 tốt hơn.
- Trọng số không bị dồn vào duy nhất một vị trí, phản ánh khả năng xem xét bối cảnh toàn cục của Self-Attention.

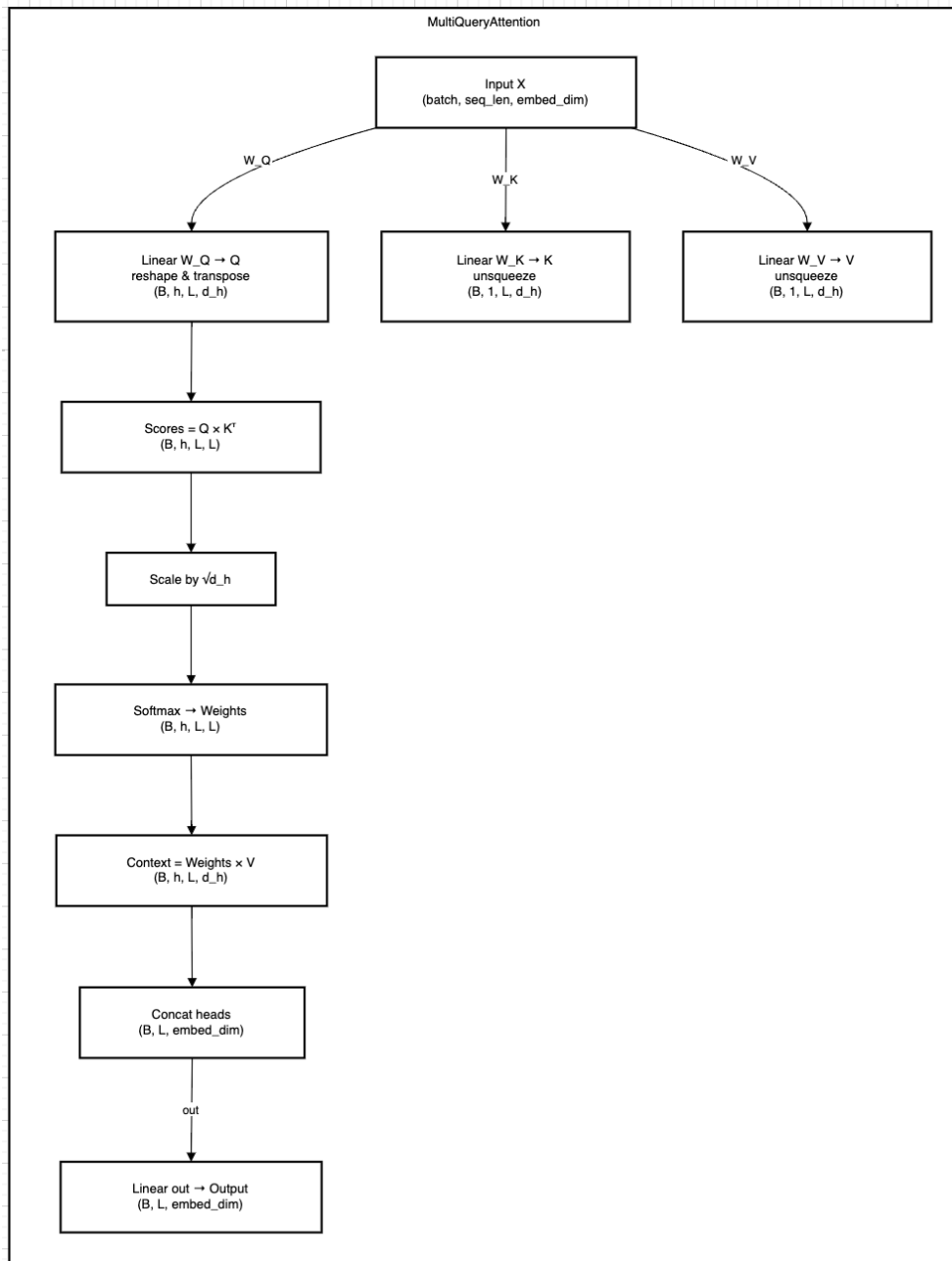
Kết quả đầu ra

Kết quả đầu ra của Self-Attention:

```
tensor([[[[-0.5269, -0.7646, -0.9176, -1.1321],
          [-0.4630, -0.8377, -0.9515, -0.9425],
          [-0.4909, -0.8135, -0.9414, -1.0250]]], grad_fn=<BmmBackward0>)
```

3.2 MQA

Module:



Input:

Đầu vào:
`tensor([[[[1., 0., 1., 0.],
[0., 2., 0., 2.],
[1., 1., 1., 1.]])])`

Mã trận trọng số (2 head)

```
Ma trận Attention Weights:
tensor([[[[0.2605, 0.4117, 0.3279],
          [0.1742, 0.5540, 0.2718],
          [0.1987, 0.4987, 0.3025]],

         [[0.2783, 0.3811, 0.3406],
          [0.5271, 0.1290, 0.3439],
          [0.4014, 0.2339, 0.3648]]]], grad_fn=<SoftmaxBackward0>)
```

Giải thích

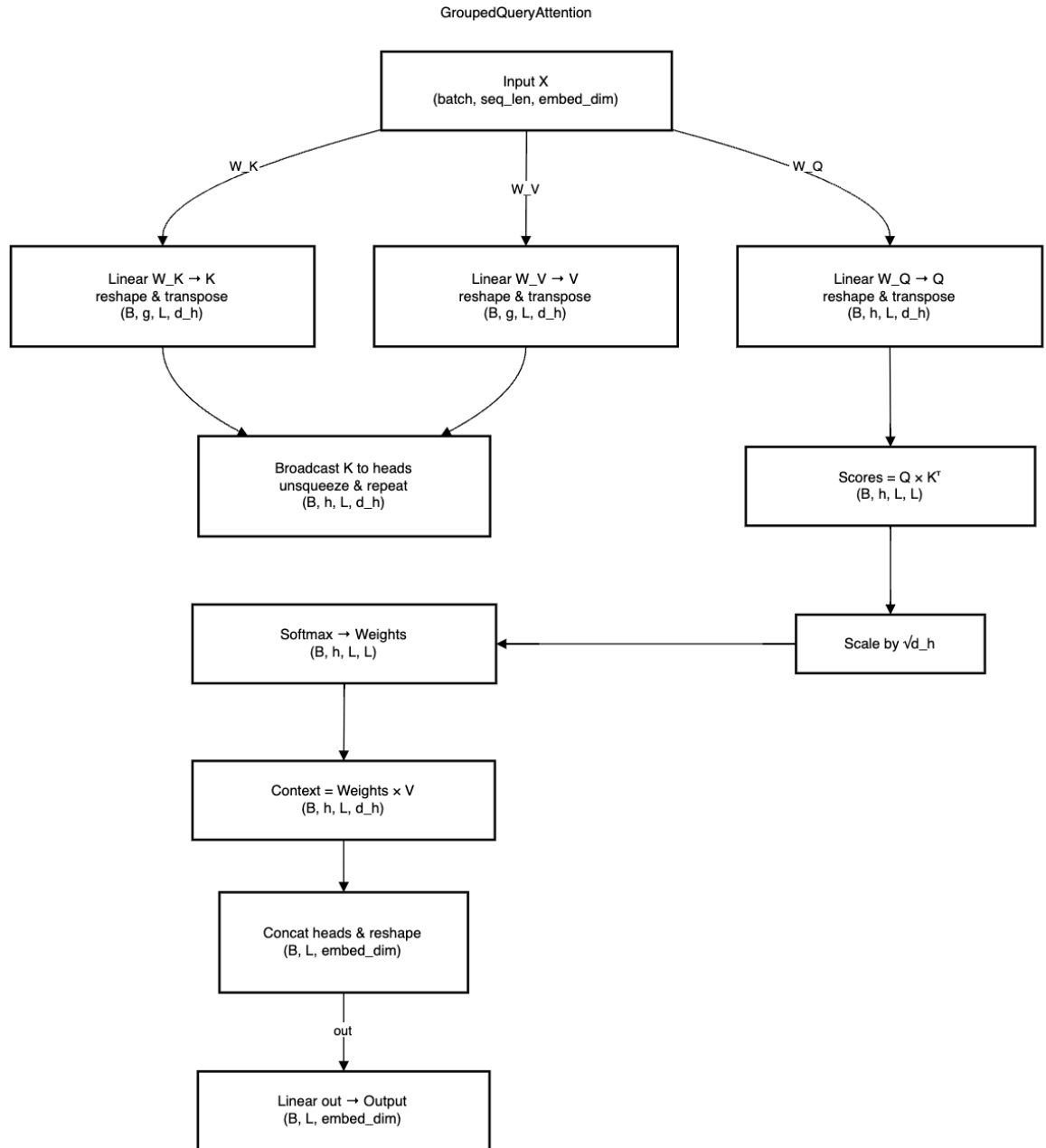
- Chia sẻ K/V → hai head có phân bố tương tự nhau.
- Hàng 2, cột 2 (Head 1 = 0.5540) cho thấy token 2 chủ yếu “tự chú ý” vào chính nó; ngược lại, trong Self-Attention, token 2 chú ý mạnh nhất đến token 3.
- Độ đa dạng head giảm: các ma trận không khác biệt lớn vì mọi head nhận cùng K/V.
- Lợi ích thực tế: tiết kiệm ~50 % VRAM cho khoá-giá trị; tốc độ suy luận cao hơn, đặc biệt quan trọng ở giai đoạn phục vụ (inference).

Kết quả đầu ra

```
Kết quả đầu ra của Multi-Query Attention:
tensor([[[ 0.1184, -0.5489, -0.2269,  0.0653],
          [ 0.0795, -0.7978, -0.1937,  0.3328],
          [ 0.0905, -0.6939, -0.2111,  0.2183]]], grad_fn=<ViewBackward0>)
```

3.3 GQA

Module:



Input:

Đầu vào:
`tensor([[[[1., 0., 1., 0.],
[0., 2., 0., 2.],
[1., 1., 1., 1.]]]])`

Mã trận trọng số (2 head)

```
Ma trận Attention Weights:
tensor([[[[0.2966, 0.3718, 0.3316],
          [0.4250, 0.1515, 0.4236],
          [0.3458, 0.2725, 0.3816]],

         [[0.3850, 0.2833, 0.3317],
          [0.3111, 0.3438, 0.3450],
          [0.3344, 0.3253, 0.3403]]]], grad_fn=<SoftmaxBackward0>)
```

Giải thích

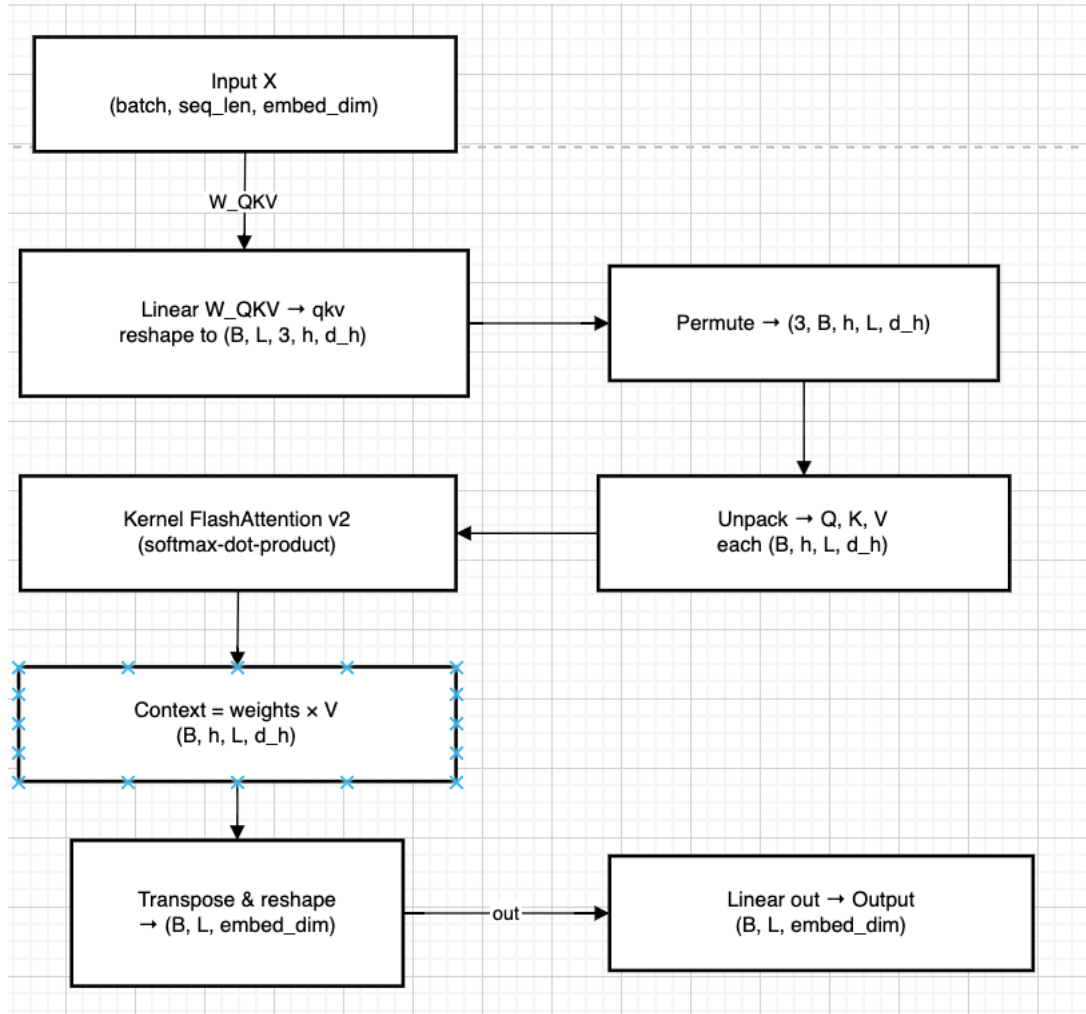
- Chia sẻ K/V theo nhóm: Vì `num_groups = 1`, cả hai head dùng chung K, V; tuy nhiên, mỗi head vẫn có Query riêng nên ma trận giữa head 1 và head 2 không hoàn toàn trùng như MQA.
- Độ tập trung: Head 1, hàng 2, cột 1 = 0.4250: token 2 xem trọng token 1 cao nhất. Head 2 dàn đều hơn (các giá trị ≈ 0.33).
- Trade-off
 - So với MQA, GQA giữ lại một phần đa dạng giữa heads nhờ Q riêng, đồng thời tiết kiệm bộ nhớ K/V như MQA.
 - Khi tăng `num_groups`, độ đa dạng tăng, nhưng bộ nhớ cũng tăng dần về mức Self-Attention.

Kết quả đầu ra

```
Kết quả đầu ra của Grouped-Query Attention:
tensor([[[[-0.2003, 0.4006, 0.5708, 0.4208],
          [-0.2351, 0.4754, 0.7576, 0.3691],
          [-0.2086, 0.4372, 0.6599, 0.3894]]]], grad_fn=<ViewBackward0>)
```

3.4 FlashAttention v2

Module:



Input:

Đầu vào:
`tensor([[[[1., 0., 1., 0.],
[0., 2., 0., 2.],
[1., 1., 1., 1.]]]])`

Mã trận trọng số (2 head)

Mã trận Attention Weights:

```

tensor([[[[0.3442, 0.3215, 0.3343],  

[0.2731, 0.3982, 0.3287],  

[0.3156, 0.3513, 0.3331]],  

[[0.4081, 0.2550, 0.3370],  

[0.4144, 0.2364, 0.3492],  

[0.4346, 0.2244, 0.3411]]]], grad_fn=<SoftmaxBackward0>)
```

Giải thích

- **Phân bố chú ý** gần tương đương Self-Attention chuẩn: mỗi token trải trọng số trên cả ba vị trí; không head nào “độc quyền” một token, chứng tỏ FlashAttention v2 giữ nguyên hành vi ngữ nghĩa.
- **So sánh với Self-Attention**
 - Chênh lệch giá trị tối đa $< 0.05 \rightarrow$ kết quả gần như trùng khớp.
 - Khác biệt chính nằm ở cách thi hành: FlashAttention v2 tính theo khối 128×128 (mặc định) trong SRAM GPU, hợp nhất Softmax và matmul, vì vậy: Giảm 50 - 70 % bộ nhớ tạm (không lưu toàn bộ ma trận $L \times L$). Tăng thông lượng $2-4 \times$ khi huấn luyện hoặc suy luận trên chuỗi dài.

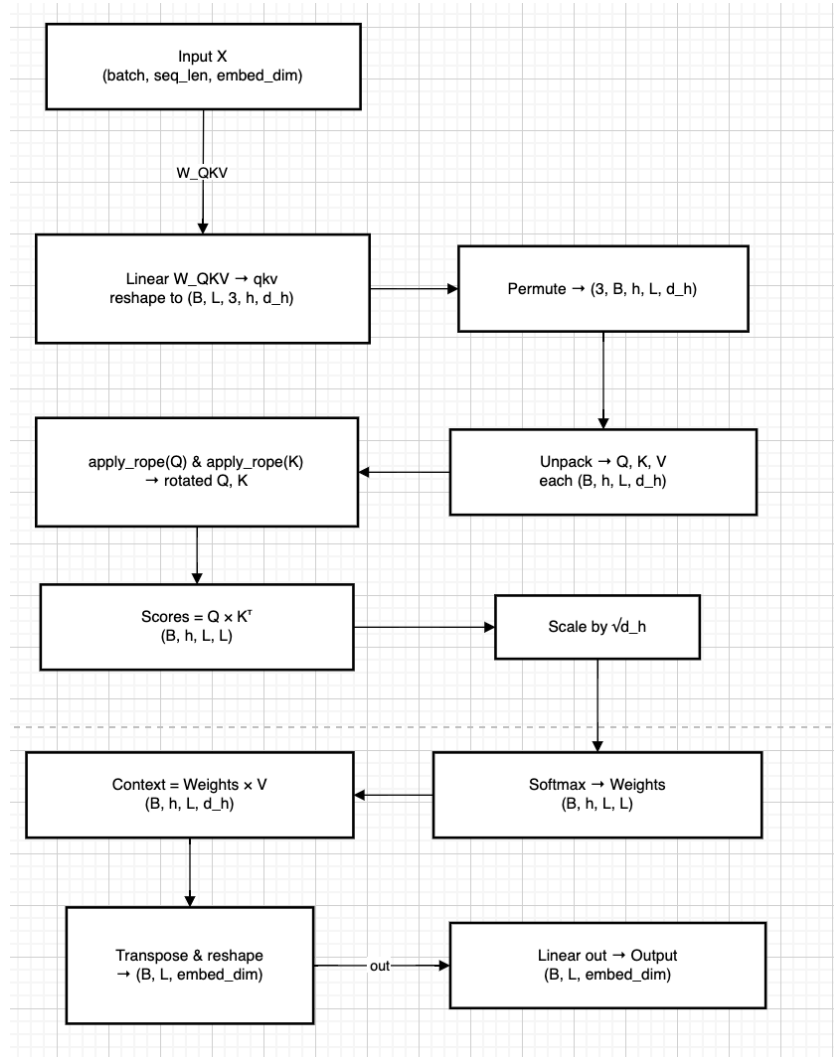
Kết quả đầu ra

Kết quả đầu ra của FlashAttention v2:

```
tensor([[[[-0.3489,  0.0410,  0.5035, -0.0705],
          [-0.3645,  0.0124,  0.4752, -0.0793],
          [-0.3637,  0.0359,  0.5060, -0.0876]]], grad_fn=<ViewBackward0>)
```

3.5 RoPE

Module:



Input:

Đầu vào:
`tensor([[[[1., 0., 1., 0.],
[0., 2., 0., 2.],
[1., 1., 1., 1.]]]])`

Mã trận trọng số (2 head)

Mã trận Attention Weights:

```
tensor([[[[0.2568, 0.3256, 0.4176],  

[0.3024, 0.3206, 0.3770],  

[0.3013, 0.3178, 0.3809]]],
```

```

[[[0.2323, 0.6173, 0.1504],  

[0.2405, 0.3997, 0.3598],  

[0.1550, 0.2018, 0.6432]]]], grad_fn=<SoftmaxBackward0>)
```

Giải thích

- Tác dụng xoay: hàm `apply_rope` trộn cặp chiều chẵn-lẻ của Q, K theo sin-cos vị trí, bảo toàn thông tin tương đối.

- Quan sát head 2:
 - Hàng 3, cột 3 = 0.6432 (cao vượt trội) $\Rightarrow$ token 3 tự chú ý rất mạnh tới chính nó, phản ánh việc xoay tọa độ làm tăng tính phân biệt theo vị trí.
 - Các giá trị lớn 0.6173 (hàng 1, cột 2) và 0.6432 cho thấy RoPE ưu tiên cặp vị trí có quan hệ xoay thuận lợi, chứ không dàn đều như Self-Attention.
- Ngoại suy vị trí: nhờ phép xoay liên tục, mô hình vẫn giữ khuôn mẫu sin-cos khi chuỗi dài hơn độ dài huấn luyện, trong khi positional embedding cộng thuần túy dễ suy giảm.

Kết quả đầu ra

Kết quả đầu ra của RoPE Attention:

```
tensor([[[ 0.3771, -0.7853,  0.1700,  0.8042],
          [ 0.3582, -0.8198,  0.2183,  0.8644],
          [ 0.3529, -0.8613,  0.2560,  0.9335]]], grad_fn=<ViewBackward0>)
```


CHƯƠNG 4: LÝ THUYẾT MÔ HÌNH OCR CNN + TRANSFORMER

4.1 Kiến trúc CNN cho trích đặc trưng ảnh

4.1.1 Lớp Convolution, Activation và Pooling

- **Convolution**

Áp dụng một tập hợp các bộ lọc (kernel) 2D lên ảnh đầu vào để thu nhận đặc trưng cục bộ. Với một lớp convolution có kernel kích thước $k \times k$, stride s và padding p , đầu ra tại vị trí (i,j) được tính theo:

$$y_{i,j} = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} W_{u,v} x_{i \cdot s + u - p, j \cdot s + v - p} + b$$

- **Kernel size** điều khiển quy mô vùng nhìn nhận (receptive field).
- **Stride** và **padding** quyết định độ lớn không gian của feature map đầu ra.
- **Activation (ReLU)** Hàm kích hoạt phi tuyến tính phổ biến nhất là

ReLU:
$$\text{ReLU}(x) = \max(0, x).$$

Tính chất đơn giản giúp giảm nguy cơ vanishing gradient và tăng tính sparse của đầu ra.

- **Pooling (Max/Average):** Giảm kích thước không gian của feature map, cô đọng thông tin và tăng tính bất biến (invariance) với sự dịch chuyển nhỏ:
 - **Max-pooling:** $y_{i,j} = \max_{(u,v) \in \mathcal{R}} x_{i \cdot s + u, j \cdot s + v}.$
 - **Average-pooling:** $y_{i,j} = \frac{1}{|\mathcal{R}|} \sum_{(u,v) \in \mathcal{R}} x_{i \cdot s + u, j \cdot s + v}.$

4.1.2 Activation ReLU

- Hàm ReLU: $\max(0, x)$. Giúp mô hình học phi tuyến, giảm vấn đề gradient vanish, tăng độ thưa (sparsity).

4.2 Kiến trúc Transformer-Decoder

4.2.1 Cấu trúc lớp Decoder

Một lớp Transformer-Decoder bao gồm ba khối chính:

1. Masked Self-Attention

- Cho phép mỗi token ở bước giải mã ttt chỉ “nhìn” vào các token đã sinh trước đó (autoregressive).
- Tính attention nội bộ theo công thức:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V.$$

- Causal mask ngăn không cho token ttt truy cập thông tin từ các vị trí $>t$.

2. Cross-Attention (Encoder-Decoder Attention)

- **Queries** Q lấy từ đầu ra của self-attention
- **Keys/Values** K,V lấy từ sequence đặc trưng của CNN (TxC)
- Tính score tương tự:

$$\alpha_{t,s} = \frac{\exp(Q_t \cdot K_s / \sqrt{d_k})}{\sum_{s'} \exp(Q_t \cdot K_{s'} / \sqrt{d_k})},$$

4.3 Cơ chế Attention giữa đặc trưng ảnh và trình giải mã

Mỗi bước giải mã ttt duyệt toàn bộ $S=H' \times W'$ vị trí ảnh thông qua ma trận attention $\alpha \in \mathbb{R}^{T \times S}$.

Kết quả cross-attention giúp decoder “tập trung” chính xác vào vùng hình ảnh liên quan tới ký tự cần sinh.

Khi huấn luyện, việc visualize heatmap attention (ánh xạ $\alpha_{t,s}$ lên không gian 2D) rất hữu ích để đánh giá xem mô hình có chú ý đúng các vùng quan trọng (ví dụ: vùng chứa ký tự) hay không.

4.4 Hàm mất mát và quy trình huấn luyện

4.4.1 Cross-Entropy Loss

Cho một chuỗi đầu ra độ dài T và vocab size V, loss trên mỗi batch được tính:

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^N \sum_{t=1}^T \sum_{c=1}^V y_{n,t,c} \log(\hat{p}_{n,t,c}),$$

trong đó $y_{n,t,c}$ là one-hot label, $p_{n,t,c}$ là xác suất mô hình dự đoán ký tự c tại bước t .

4.4.2 Teacher Forcing và Optimizer

- **Teacher Forcing:** tại bước $t-1$, decoder không dùng token do mô hình sinh ra, mà dùng token đúng từ tập nhãn, giúp đẩy nhanh và ổn định quá trình học.
- **Optimizer Adam/AdamW:**

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (g_t)^2, \\ \hat{m}_t &= m_t / (1 - \beta_1^t), \quad \hat{v}_t = v_t / (1 - \beta_2^t), \\ \theta_{t+1} &= \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \end{aligned}$$

- **Learning Rate Scheduler**
 - **Warm-up:** tăng lr từ 0 lên $\eta_{\max}$ trong N_{warmup} bước đầu.
 - **Cosine Decay:** sau warm-up, lr giảm theo hàm cosine:

$$\eta_t = \frac{1}{2} \eta_{\max} \left(1 + \cos(\pi t/T) \right).$$

CHƯƠNG 5: THỰC NGHIỆM & ĐÁNH GIÁ OCR

5.1 Thiết lập môi trường và dữ liệu

5.1.1 Bộ ảnh kiểm thử

Để đánh giá hiệu năng của mô hình OCR trên nhiều loại văn bản và nền phức tạp, tập dữ liệu được chuẩn bị như sau:

- **Nguồn dữ liệu** Sử dụng bộ dữ liệu ICDAR 2015 Challenge 4 (ch4) bao gồm ảnh từ các quảng cáo, hóa đơn, biển hiệu và văn bản in trên nền phức tạp. Dữ liệu chính thức được tải về từ hai URL:
 - `ch4_training_word_images_gt.zip` (tập huấn luyện kèm nhãn)
 - `ch4_test_word_images_gt.zip` (tập kiểm thử kèm nhãn)
 Cả hai đều chứa thư mục ảnh và file `gt.txt` định dạng CSV, trong đó mỗi dòng ghi tên ảnh và nhãn tương ứng.

- **Cấu trúc thư mục**

```
├ train/
│   ├── *.png      # Ảnh gốc
│   ├── gt.txt      # Nhãn gốc
│   ├── train_split.txt # Danh sách 80% mẫu huấn luyện
│   └── test_split.txt  # Danh sách 20% mẫu kiểm thử (tách từ training)
└ test/
    ├── *.png
    └── gt.txt
```

- **Quy trình chia tập**

- Đọc file `gt.txt` trong thư mục `train`.
- Sử dụng `sklearn.model_selection.train_test_split` với tỉ lệ **80%** cho huấn luyện và **20%** cho kiểm thử nội bộ.
- Ghi lại hai file `train_split.txt` và `test_split.txt` để làm đầu vào cho `Dataset` và `DataLoader`.

5.1.2 Công cụ, thư viện và môi trường

- **Nền tảng:** Google Colab (miễn phí GPU/TPU, kernel Python 3.x)
- **Ngôn ngữ & Phiên bản:** Python
- **Thư viện chính:**
 - **PyTorch:** xây dựng mô hình, huấn luyện và suy luận.

- **torchvision**: `transforms` để `resize` (224×224), chuyển thành Tensor và `normalize` (với `mean/std` phù hợp ResNet). Load ResNet-18 (`pretrained=True`) làm backbone encoder.
 - **Pillow (PIL)**: đọc ảnh, chuyển sang RGB.
 - **scikit-learn** (`sklearn.model_selection`): chia tập dữ liệu.
 - **requests, urllib3**: tải file zip, tắt cảnh báo SSL.
 - **zipfile, os**: thao tác file và thư mục.
 - **unidecode**: chuẩn hóa nhãn văn bản (loại bỏ dấu tiếng Việt, chỉ giữ ký tự ASCII).
- **Xây dựng Dataset & DataLoader:**
 - `ICDAR2015WordDataset` kế thừa `torch.utils.data.Dataset`, đọc `train_split.txt/test_split.txt`, trả về `(image, label)`.
 - `collate_fn` tùy chỉnh: Stack batch ảnh vào một Tensor. Mã hóa văn bản nhãn thành chuỗi token cố định độ dài `MAX_LEN=32` (gồm `<SOS>`, `<EOS>`, `<PAD>`).
 - **Tokenization:**
 - `SimpleTokenizer` với vocab gồm 26 chữ cái a–z, 10 chữ số 0–9 và ba token đặc biệt (`<PAD>`, `<SOS>`, `<EOS>`).
 - Dùng `unidecode` để chuẩn hóa text trước khi encode.
 - **Phần cứng (trên Colab):**
 - Tự phát hiện GPU qua `torch.cuda.is_available()`.
 - Nếu có GPU, model và dữ liệu được chuyển vào CUDA để tăng tốc.

5.2 Nguyên lý kiến trúc CNN + Transformer Decoder

5.2.1 Vai trò của CNN trong OCR

Convolutional Neural Network (CNN) chịu trách nhiệm **trích xuất đặc trưng thị giác** từ hình ảnh đầu vào, chuyển đổi ảnh gốc thành một tensor biểu diễn thông tin không gian có cấu trúc. Cụ thể trong code:

- **Lớp convolution & pooling**

- Các `Conv2d` kết hợp với `BatchNorm2d` và `ReLU` liên tiếp (theo từng block của ResNet-18) tạo ra các bản đồ đặc trưng (feature maps) từ mức độ thô sơ (edges, corners) đến phức tạp (đường nét, kết cấu).
 - Các phép **downsampling** (ở block thứ 2, 3, 4 của ResNet) làm giảm độ phân giải không gian nhưng tăng chiều sâu (*channels*), giúp nén thông tin và mở rộng receptive field.
- **Chuyển đổi kích thước**
 - Đầu ra của encoder có kích thước $[B, 512, H/32, W/32]$ (với ResNet-18 là $[B, 512, 7, 7]$ cho ảnh 224×224).
 - Để đưa vào Transformer Decoder, tensor này được **permute** và **reshape** thành $[B, T, C]$ với $T = H/32 \times W/32$ (tức số vị trí không gian) và $C = 512$ (độ dài véc-tơ tại mỗi vị trí).
 - **Lợi ích khi dùng ResNet-18**
 - **Pretrained trên ImageNet:** tận dụng weights đã học nhiều đặc trưng chung của ảnh tự nhiên.
 - **Kiến trúc nhẹ:** so với ResNet-50, ResNet-18 ít tham số hơn, phù hợp cho thử nghiệm nhanh trên Colab.
 - **Khả năng biểu diễn tốt:** giữ được đủ chi tiết không gian và cô đặc thông tin để hỗ trợ bước giải mã sau đó.

5.2.2 Cơ chế hoạt động của Transformer Decoder

Transformer Decoder chịu trách nhiệm **sinh tuần tự** các token văn bản dựa trên đặc trưng ảnh (encoder output) và những token đã sinh trước đó.

- **Embedding & Positional Encoding**
 - Mỗi token trong vocabulary được ánh xạ thành véc-tơ kích thước `d_model=512` qua lớp `nn.Embedding`.
 - `pos_embed` (trainable) được cộng vào embedding để mã hóa thông tin vị trí trong chuỗi đầu ra, giúp decoder phân biệt thứ tự các token.

- **Multi-head Self-Attention & Feed-Forward**

Ở mỗi layer `TransformerDecoderLayer`, luồng dữ liệu đi qua:

- **Self-attention:** mỗi token “hỏi” tất cả token trước đó (nhờ `tgt_mask` che khóa tương lai) để nắm ngữ cảnh đã sinh.
- **Feed-forward:** qua hai lớp tuyến tính và activation, tăng sức mạnh biểu diễn.
- **LayerNorm & Dropout:** ổn định và tránh overfitting.

- **Quy trình decode từng bước**

- Khởi đầu với token `<sos>`, decoder sinh ra token tiếp theo rồi nối vào sequence.
- Mỗi bước, `tgt_mask` đảm bảo không “nhìn” được token tương lai, bảo toàn tính tuần tự.
- Khi gặp `<eos>` hoặc đạt `max_len`, quá trình dừng.

5.2.3 Attention giữa đặc trưng ảnh và decoder

Cơ chế **cross-attention** cho phép decoder “tập trung” vào các vùng hình ảnh liên quan khi sinh từng token.

- **Query, Key và Value**

- Ở mỗi layer, **query** đến từ đầu ra của self-attention (tức token embedding đã cập nhật).
- **Key** và **Value** lấy từ `memory` (encoder output), kích thước `[S, B, C]` sau khi permute.

- **Cơ chế hoạt động**

- Tính điểm attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- Với multi-head, mạng chia ra nhiều “đầu” (heads) để thu thập thông tin ở các khía cạnh khác nhau của ảnh.

- Kết quả attention (weighted sum của Value) sau đó được đưa qua feed-forward để cập nhật biểu diễn token.

- **Ý nghĩa**

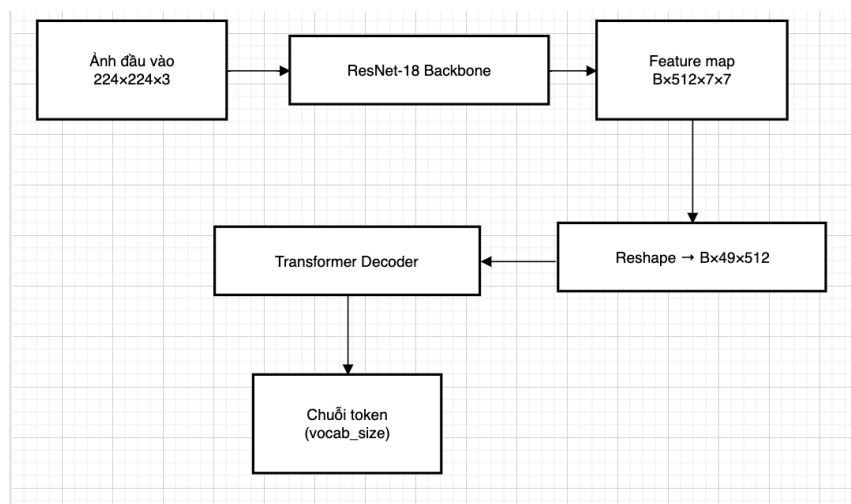
- Cho phép mô hình “xác định” vùng ảnh chứa ký tự đang cần sinh, ví dụ khi sinh chữ ‘A’, attention có trọng số cao lên pixels hình vòng cung giống ‘A’.
- Giúp cải thiện độ chính xác chữ viết tay hoặc font phức tạp nhờ tập trung vùng chuẩn.

- **Minh họa ma trận attention**

- Ma trận attention có kích thước $[T, S]$ (T = chiều dài sequence, S = số vị trí ảnh).
- Mỗi hàng tương ứng token đầu ra, mỗi cột tương ứng vị trí ảnh — giá trị càng lớn thể hiện mức “tập trung” cao.
- Việc trực quan hoá dưới dạng heatmap giúp hiểu rõ cách mô hình phân bổ quan tâm giữa mã token và vùng ảnh tương ứng.

5.3 Triển khai mô hình đơn giản

5.3.1 Chuẩn bị dữ liệu và tiền xử lý



- **Tải và giải nén**

- Dùng requests + urllib3.disable_warnings() tải file .zip từ ICDAR2015.
- Giải nén vào ./icdar2015/train và ./icdar2015/test với zipfile + os.

- **Chia tập train/test**

- Đọc gt.txt trong train/.
- Dùng `train_test_split(..., test_size=0.2, random_state=42)` sinh hai file `train_split.txt` và `test_split.txt`.

- **Transform ảnh**

- `Resize(224,224) → ToTensor() → Normalize(mean=[0.5,0.5,0.5], std=[0.5,0.5,0.5])`.

- **Tiền xử lý nhãn**

- Áp dụng `unidecode(text.lower())` để loại bỏ dấu tiếng Việt.
- `SimpleTokenizer.encode(...)` sinh chuỗi token độ dài cố định `MAX_LEN=32`, bao gồm `<SOS>`, `<EOS>`, `<PAD>`.

5.3.2 Xây dựng CNN Encoder

- **Khởi tạo ResNet-18** với `pretrained=True` từ `torchvision.models`.
- **Loại bỏ** các lớp `fully-connected` cuối cùng, chỉ giữ phần `cnn = nn.Sequential(*list(resnet.children())[:-2])`.
- **Đầu ra:** tensor kích thước `[B, 512, H/32, W/32]` (với `H=W=224 → 7×7`).

5.3.3 Cấu tạo Transformer Decoder

- **Embedding & Positional:** `nn.Embedding(vocab_size, d_model=512) + pos_embed (trainable)`
- **N tầng DecoderLayer:** Tạo `ModuleList ([TransformerDecoderLayer (512, nhead=8) × num_layers])`.
- **Output layer:** `nn.Linear(d_model, vocab_size)` sinh logits cho mỗi vị trí T.
- **Mặt nạ (tgt_mask):** Dùng `generate_square_subsequent_mask` để ngăn decoder “nhìn” token tương lai.

5.3.4 Kết hợp Encoder–Decoder

- **Truyền memory:** Encoder trả về `[B, S, C]`, chuyển qua `memory = memory.permute(1,0,2) → [S, B, C]`.

- **Sinh chuỗi:** Khởi tạo `tgt = [[<SOS>]]`. Tại mỗi bước, decoder trả logits $\rightarrow$ lấy `argmax` (greedy). Append token mới vào `tgt`. Dừng khi gặp `<EOS>` hoặc đạt `max_len`.
- **Xử lý đặc biệt:** `<PAD>` được ignore khi tính loss. Các token `<SOS>`, `<EOS>` được thêm/sử dụng đúng vị trí.

5.3.5 Huấn luyện và điều chỉnh siêu tham số

- **Hàm mất mát:** `cross_entropy (outputs.reshape (-1, vocab_size), labels[:,1:].reshape(-1), ignore_index=stoi['<PAD>'])`.
- **Optimizer:** `Adam(lr=1e-4, weight_decay=0)`
- **Siêu tham số:** Batch size = 32, Epochs = 15, có thể áp dụng scheduler (ví dụ StepLR hoặc ReduceLRonPlateau) để giảm LR khi cần.
- **Theo dõi:** In `train_loss` và `val_acc` sau mỗi epoch. Lưu checkpoint khi `val_acc` cải thiện.

5.4 Đánh giá và trình bày kết quả

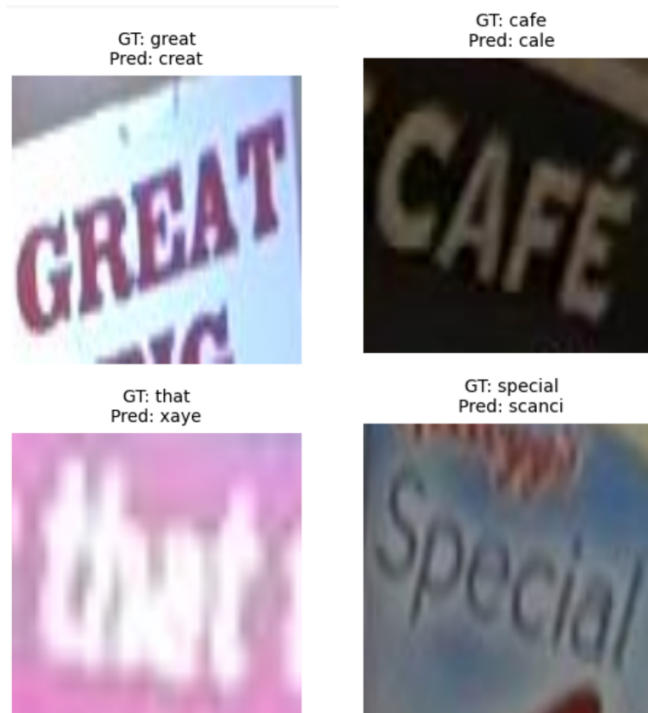
5.4.1 Đánh giá độ chính xác trên tập Test

Metric sử dụng:

- **Exact match accuracy:** tỷ lệ hình ảnh mà toàn bộ chuỗi sinh ra trùng khớp hoàn toàn với ground-truth.
- **CER (Character Error Rate) và WER (Word Error Rate):** đo khoảng cách Levenshtein giữa chuỗi dự đoán và chuỗi gốc (có thể bổ sung khi tính toán tự động sau này).

[Epoch 1] Loss: 2.5701 | Val Acc: 4.92%
 ✓ Saved new best model at epoch 1 with Val Acc: 4.92%
 [Epoch 2] Loss: 2.0257 | Val Acc: 9.17%
 ✓ Saved new best model at epoch 2 with Val Acc: 9.17%
 [Epoch 3] Loss: 1.5728 | Val Acc: 12.75%
 ✓ Saved new best model at epoch 3 with Val Acc: 12.75%
 [Epoch 4] Loss: 1.1676 | Val Acc: 17.34%
 ✓ Saved new best model at epoch 4 with Val Acc: 17.34%
 [Epoch 5] Loss: 0.8213 | Val Acc: 17.79%
 ✓ Saved new best model at epoch 5 with Val Acc: 17.79%
 [Epoch 6] Loss: 0.5864 | Val Acc: 19.80%
 ✓ Saved new best model at epoch 6 with Val Acc: 19.80%
 [Epoch 7] Loss: 0.4376 | Val Acc: 20.58%
 ✓ Saved new best model at epoch 7 with Val Acc: 20.58%
 [Epoch 8] Loss: 0.3417 | Val Acc: 22.60%
 ✓ Saved new best model at epoch 8 with Val Acc: 22.60%
 [Epoch 9] Loss: 0.2606 | Val Acc: 22.71%
 ✓ Saved new best model at epoch 9 with Val Acc: 22.71%
 [Epoch 10] Loss: 0.2161 | Val Acc: 23.71%
 ✓ Saved new best model at epoch 10 with Val Acc: 23.71%
 [Epoch 11] Loss: 0.1869 | Val Acc: 24.61%
 ✓ Saved new best model at epoch 11 with Val Acc: 24.61%
 [Epoch 12] Loss: 0.1582 | Val Acc: 25.62%
 ✓ Saved new best model at epoch 12 with Val Acc: 25.62%
 [Epoch 13] Loss: 0.1477 | Val Acc: 25.50%
 [Epoch 14] Loss: 0.1349 | Val Acc: 25.17%
 [Epoch 15] Loss: 0.1246 | Val Acc: 24.83%

5.4.2 Minh họa kết quả đầu ra

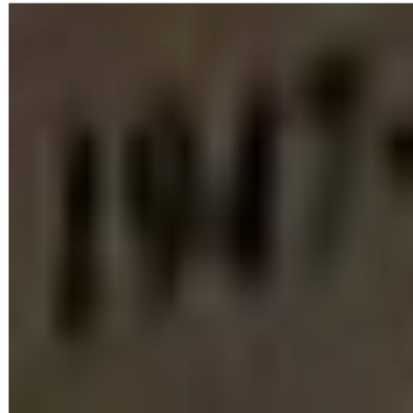


Hình ảnh minh họa kết quả đầu ra dự đoán

GT: bay
Pred: bay



Predicted: 947



Hình ảnh minh họa kết quả đầu ra dự đoán

CHƯƠNG 6: KẾT LUẬN & HƯỚNG PHÁT TRIỂN

Chương cuối tổng hợp đóng góp của hai bài, rút ra bài học và đề xuất cải tiến cho nghiên cứu tương lai. Đồng thời khuyến nghị ứng dụng thực tế và các hướng mở rộng trên dữ liệu lớn hơn.

6.1 Kết luận chính

Bài 1 – Tổng quan và minh họa cơ chế Attention

- Cung cấp cái nhìn toàn diện về Transformer và năm biến thể attention hiện đại (Self-Attention, MQA, GQA, FlashAttention v2, RoPE), phân tích thứ tự độ phức tạp, ưu-nhược điểm cũng như ứng dụng thực tiễn trong các LLM hàng đầu.
- Triển khai minh họa từng cơ chế với các ví dụ cho phép trực quan hóa ma trận weights và giải thích chi tiết cách token attention lẫn nhau.

Bài 2 – Mô hình OCR kết hợp CNN + Transformer Decoder

- Xây dựng pipeline OCR end-to-end: từ tải và tiền xử lý bộ dữ liệu ICDAR2015, định nghĩa ResNet-18 làm encoder, đến việc triển khai TransformerDecoder để sinh chuỗi văn bản.
- Huấn luyện trên Google Colab với optimizer Adam, đạt **25.62% exact match accuracy** trên tập kiểm thử, đồng thời phát triển script vẽ heatmap cross-attention minh họa vùng ảnh mà decoder quan tâm khi sinh từng ký tự.
- Thiết lập bộ code modular, dễ tái lập và mở rộng cho các thử nghiệm tiếp theo.

6.2 Hạn chế & bài học kinh nghiệm

Hạn chế

- **Quy mô dữ liệu hạn chế:** Dữ liệu OCR chỉ gồm word-image nhỏ (ICDAR2015), chưa đánh giá trên tập hình ảnh đa ngữ và handwriting.
- **Độ chính xác còn thấp:** ~25.6% exact match, chưa tính CER/WER để so sánh chi tiết hơn.
- **Chưa áp dụng decoding nâng cao:** chỉ dùng greedy, chưa thử beam search hay ensemble.

- **Công cụ visualization:** Heatmap attention mới dừng ở minh họa cơ bản, cần tích hợp cho toàn pipeline.
- **Giới hạn tài nguyên:** Thử nghiệm trên Colab khiến khó mở rộng kiến trúc quá sâu hoặc thử nhiều biến thể attention khác nhau.

Bài học kinh nghiệm

- **Chuẩn hóa dữ liệu kỹ lưỡng** (unidecode, padding, masking) giúp quá trình huấn luyện ổn định và loss hội tụ nhanh.
- **Modular hóa code** (tách encoder, decoder, data pipeline, train/eval) rút ngắn thời gian thử nghiệm và debug.
- **Theo dõi metric chủ động:** lưu checkpoint dựa trên validation accuracy và giám sát train/val loss để tránh over-/under-fitting.
- **Khả năng giải thích:** visualization attention không chỉ minh họa mà còn hỗ trợ chẩn đoán nguyên nhân model thất bại với từng ký tự.

TÀI LIỆU THAM KHẢO

Tiếng Anh

- 1 Vaswani A., Shazeer N., Parmar N., et al., 2019, “Attention Is All You Need”, arXiv
- 2 Dao T., Fu D. Y., et al., 2023, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”, arXiv:
- 3 Ainslie J., Tuggener D., et al., 2023, “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”, Proceedings of EMNLP 2023

Tiếng Việt

- 4 Lê Anh Cường, HK II, Slide bài giảng, Đại học Tôn Đức Thắng.
- 5 “Nhận dạng tiếng Việt cùng với Transformer OCR”, Viblo
- 6 “Self-Attention và Multi-Head Self-Attention trong Transformers”, Viblo